



ORACLE

Création d'une base de données

**Création et Exploitation
de bases de données
420-315-AL**

Auteur :

Karim Mihoubi
Département d'informatique
Cégep André-Laurendeau
1111 Lapierre, Montréal (arr. LaSalle),
H8N 2J4
Tél: (514) 364-3320 [6162]
Karim.mihoubi@claurendeau.qc.ca



ORACLE

Les bases du SQL

**Création et Exploitation
de bases de données
420-315-AL**

Auteur :

Karim Mihoubi
Département d'informatique
Cégep André-Laurendeau
1111 Lapierre, Montréal (arr. LaSalle),
H8N 2J4
Tél: (514) 364-3320 [6162]
Karim.mihoubi@claurendeau.qc.ca



Objectifs de la leçon

ORACLE

- Le *SQL* déclaratif : présentation
 - Les familles de langage
 - Éléments syntaxiques
- Langage de *d*éfinition de *d*onnées (LDD)
 - Création de *table*
 - Notions de *contraintes*
 - Contraintes domaine



Éléments syntaxiques

ORACLE

**Le SQL déclaratif contient environ 40 énoncés
subdivisés en quatre catégories**

- **LDD – (DDL)**
 - **Langage de définition de données**
 - Pour les aspects structurels: créer, modifier ou détruire un objet
- **LMD – (DML)**
 - **Langage de manipulation de données**
 - Pour gérer le contenu des objets par ajouts, mises à jour ou destructions des contenus des objets; aussi requêtes sur les objets
- **LCD – (DCL)**
 - **Langage de Contrôle**
 - Pour la gestion des droits d'accès aux objets et tout autre fonction de gestion générale permise
- **LCT – (TCL)**
 - **Langage de Contrôle des transactions**
 - Pour la gestion des transactions qui englobent les commandes des trois premières sections



Diagrammes syntaxiques SQL

ORACLE

- Les **diagrammes syntaxiques** sont utilisés afin de résumer les règles d'utilisation de chaque énoncé **SQL**
- Dans le **diagramme syntaxique**, les mots en **MAJUSCULES** sont les **mots-clés**
- Les mots en *italique* sont des *variables* que l'utilisateur spécifie
- Les **alternatives** sont illustrées par les **divers chemins**
- Un choix par défaut est souligné



Diagrammes syntaxiques SQL

ORACLE

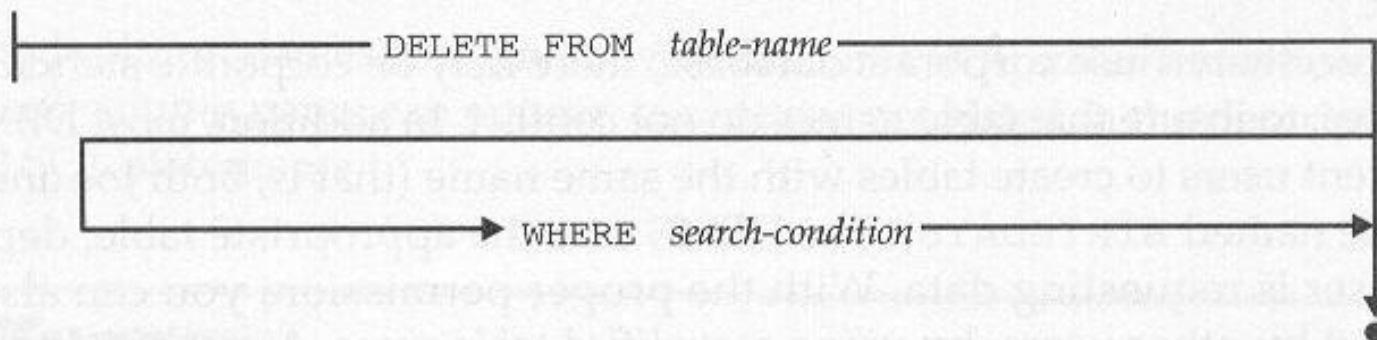
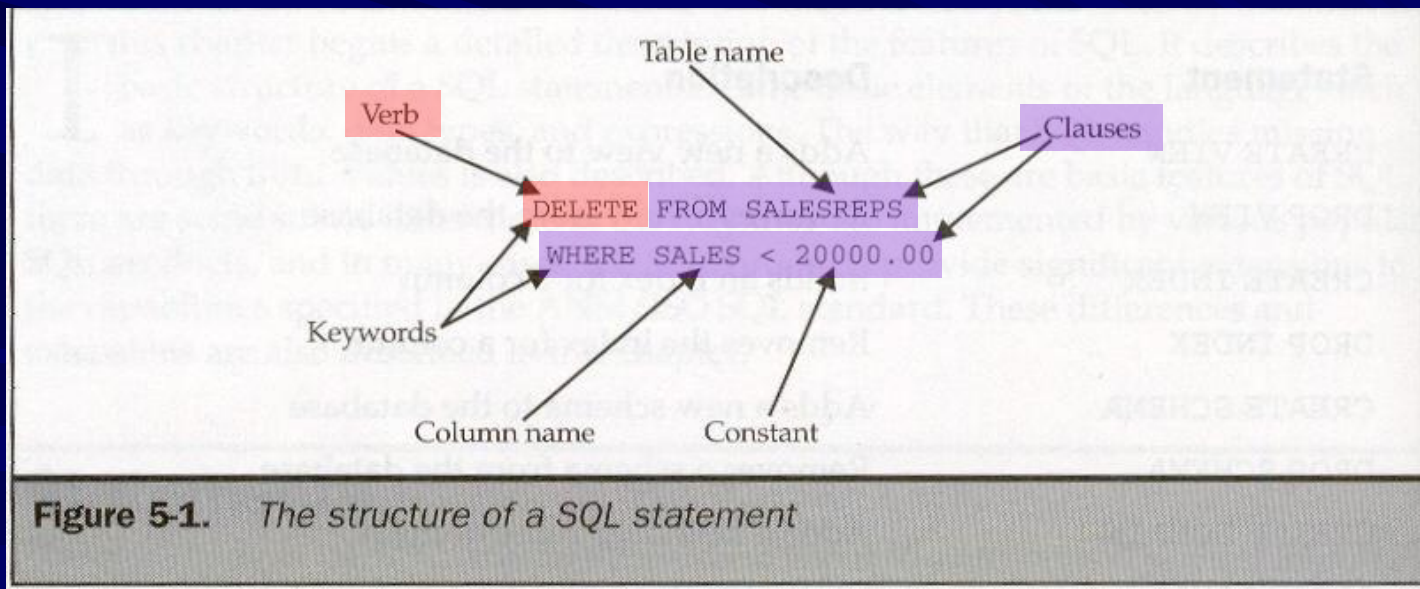


Figure 5-2. A sample syntax diagram



Syntaxe d'un énoncé SQL ORACLE

- Chaque énoncé est identifié par un **verbe** qui est un **mot-clé** en **SQL**
- L'énoncé contient une ou plusieurs **clauses**
- Chaque **clause** est identifiée par un **mot-clé**





Noms des identifiants

ORACLE

- Avec **ORACLE**, un nom d'identifiant (table, index, variable, colonne, etc.) doit respecter les règles suivantes:
 - Caractères permis: lettres, chiffres, \$, # et _
 - Doit débuter avec un caractère alphabétique
 - **30 caractères au maximum**, sauf...
 - Nom de la base de données → 8 caractères
 - Nom de liens → 128 caractères
 - Nom → insensible à la casse
 - Aucun guillemet imbriqué
- Les règles complètes sont disponibles dans **SQL Reference**



Noms qualifiés et schéma

ORACLE

- À moins d'avis contraire, **ORACLE** présume que les tables nommées font partie du **schéma de l'utilisateur** en cours
- On peut se référer à une table d'un autre usager en qualifiant le nom avec le nom de schéma de cet usager :
HR.EMPLOYEES
- On peut aussi se référer à un attribut de la table d'un autre schéma :
HR.EMPLOYEES.hire_date



Langage de *D*éfinition de *D*onnées

ORACLE

- Les trois principaux énoncés du langage de description de données (**LDD**) sont :
 - **CREATE**,
 - **DROP**
 - **ALTER**
- **Remarque** : Il est permis de créer, modifier ou détruire des objets même pendant que la base de données (instance et schéma) continuent d'être active.

Chapitre 01 Livre Christian Soutou (P. 21-41)



Ordre de création de Table

ORACLE

- L'énoncé **CREATE TABLE** sert à créer une **nouvelle table** qui sera stockée dans le **schéma** en cours d'utilisation. (**Pages 39-42**)
- Pour créer une nouvelle table, vous devez disposer du privilège **CREATE TABLE**
- L'administrateur de la BD utilise le SQL et son **langage de contrôle des données** (LCD) pour accorder ce privilège aux utilisateurs.



Ordre de création de Table

ORACLE

- La commande CREATE doit spécifier au minimum:
 - Nom de la table
 - Pour chaque colonne, son **nom**, son **type** et sa **longueur**
 - **Le type** de la colonne peut être un **type Oracle** ou un type implicite **ANSI**



Règles de nommage

ORACLE

- Pour faciliter la reconnaissance et la manipulation des divers objets de la base, il est recommandé d'utiliser un modèle de nomenclature
- **Table** → **Schéma_Nom_Suffixe**
 - **Schéma** → Schéma relationnel (3 car.)
 - **Nom** → Nom de la *table actuelle*
 - **Suffixe** → Contraction du nom de la table (3 car.)
- **Attribut** → **Préfixe_Nom**
 - **Préfixe** → Suffixe de la table actuelle (3 car.)
 - **Nom** → Nom de l'attribut



Table - Contraintes

ORACLE

- Niveaux de description d'une contrainte
 - **Domaine** : contrainte qui s'assure que la colonne ne peut prendre qu'une valeur dans le domaine de valeurs.
 - **Table** : contrainte qui porte sur une colonne, une ligne ou sur toute la table
 - **Assertions** : contraintes plus complexes qui peuvent porter sur plus qu'une table



Table - Contraintes

ORACLE

- Lors de la création de la table, on décrit également les **contraintes** à prendre en compte par le système. **Pourquoi ?**
 - Pour **assurer la cohérence fonctionnelle** des relations entre les tables (clés primaires, clés étrangères)
 - Pour s'assurer que les données saisies correspondent bien aux limites de l'univers que l'on modélise (**CHECK**)
 - Pour prendre en compte le fonctionnel applicatif. Contraintes souvent complexes (**assertion, déclencheur**).



Contraintes - Table

ORACLE

- Parmi les **contraintes de niveau table** on retrouve :
 - Valeur par défaut
 - Valeur **NULL** interdite

```
CREATE TABLE [schéma.]nomTable
( colonne1 type1 [DEFAULT valeur1] [NOT NULL]
[, colonne2 type2 [DEFAULT valeur2] [NOT NULL] ]
[CONSTRAINT nomContraintel typeContraintel]...) ;
```

Pages 39-42 Livre C. Soutou



Contraintes - Table

ORACLE

- La contrainte de niveau colonne **NULL** sert à spécifier si un champ peut contenir un **NULL**
- La contrainte **DEFAULT** propose une **valeur par défaut** si l'utilisateur ne spécifie aucune valeur

```
CREATE TABLE OFFICES
(OFFICE INTEGER NOT NULL,
 CITY VARCHAR(15) NOT NULL,
 REGION VARCHAR(10) NOT NULL DEFAULT 'Eastern',
 MGR INTEGER DEFAULT 106,
 TARGET MONEY DEFAULT NULL,
 SALES MONEY NOT NULL DEFAULT 0.00)
```



Contraintes - Table

ORACLE

- La valeur spéciale **NULL** est utilisée pour indiquer que la valeur d'un attribut est **inconnue (indéterminée)**
- **NULL** ne signifie pas 0 ou même une chaîne vide.
- Tester si NULL se fait de la sorte :
IF attribut IS NULL



Contraintes de domaine

ORACLE

- **Définition** : contrainte qui se construit sur la base des types de données que reconnaît le SQL 2 (CHAR, VARCHAR, DATE, INT, etc.) auquel on rajoute :
 - une valeur par défaut si besoin est,
 - 0, 1 ou plusieurs contraintes CHECK

```
CREATE DOMAIN Note AS DECIMAL(4,2) CHECK (VALUE BETWEEN 0.0 AND 20.0);  
  
CREATE DOMAINE D_POURCENT AS  
    FLOAT DEFAULT 0.0 CHECK (VALUE BETWEEN 0.0 AND 100.0);
```



Types de données



- Chaque fabricant possède ses propres types de données qui s'inspirent du standard **ANSI**
- Parmi les types de données que propose le SGBD **ORACLE** on retrouve :

CHAR → Chaînes de caractères fixes de 2000 octets

VARCHAR2 → Chaînes de caractères de longueur variable de moins de 4000 caractères

NUMBER → Valeurs numériques – entiers, décimales et flottants de taille variable avec précision de 38 chiffres

CLOB → Chaines de caractères de taille variable pouvant aller jusqu'à 128 Téra octets

Les valeurs numériques peuvent être négatives et positives. Exemple : NUMBER (6,2) peut stocker toute valeur comprise entre +9999,99 et -9999,99



Types de données



- Parmi les types de données les plus utilisés pour stocker une donnée :
- De type **binaire** (exemples : multimédia: JPG, WAV, MP3, etc.) :
 - RAW** → taille variable de maximum 2000 octets
 - BLOB** → taille variable pouvant aller jusqu'à 128 Téra octets
- De type **Date** pour stocker **une date** ou une **heure**:
 - DATE** → permet de stocker une valeur de plusieurs siècles en secondes entières, mais pas les fractions de seconde
 - TIMESTAMP** → extension du type de données DATE qui, lui autorise des fractions de seconde
 - INTERVAL**



Types DATE (P. 44)

ORACLE

- **TIMESTAMP** : type de données utilisé (Oracle, MySQL) pour stocker les fractions de secondes en tenant compte du fuseau horaire (GMT + ou moins ...). Voir page 29, 47
 - Pour savoir quelle est la configuration actuelle:
`SELECT * FROM NLS_SESSION_PARAMETERS;`
 - `ALTER SESSION SET NLS_TIMESTAMP_FORMAT='DD-MON-YYYY HH24:MI:SS.FF';`
- **INTERVAL** : type Oracle, permet de déclarer des durées (5 années, 2 mois). **P613 Livre**



Constantes



- Les dates sont prise en compte **implicitement** par le format par défaut (NLS_DATE_FORMAT), ou en appliquant en conversion explicite en utilisant la fonction de conversion **TO_DATE()**
- Il y a quelques **fonctions** « **système** » comme **SYSDATE**, qui permettent de retrouver la date courante du système



EXEMPLE - TIMESTAMP

ORACLE

```
DROP TABLE journal_bancaires;
CREATE TABLE journal_bancaires (
  id_transact  NUMBER,
  date_transact  TIMESTAMP
);

-- Conversion explicite du format du type TIMESTAMP pour la session

ALTER SESSION SET NLS_TIMESTAMP_FORMAT = 'DD-MM-YYYY HH24:MI:SS.FF6';

-- Insertion de données de type TIMESTAMP dans la table "journal_bancaires"

INSERT INTO journal_bancaires VALUES (1, '10/06/2018 10:52:29.123456');
INSERT INTO journal_bancaires VALUES (2, sysdate);
INSERT INTO journal_bancaires VALUES (3, SYSTIMESTAMP);
```

ID_TRANSACTION	DATE_TRANSACTION
1	10-06-2018 10:52:29.123456
2	02-02-2019 14:53:34.000000
3	02-02-2019 14:53:34.154000

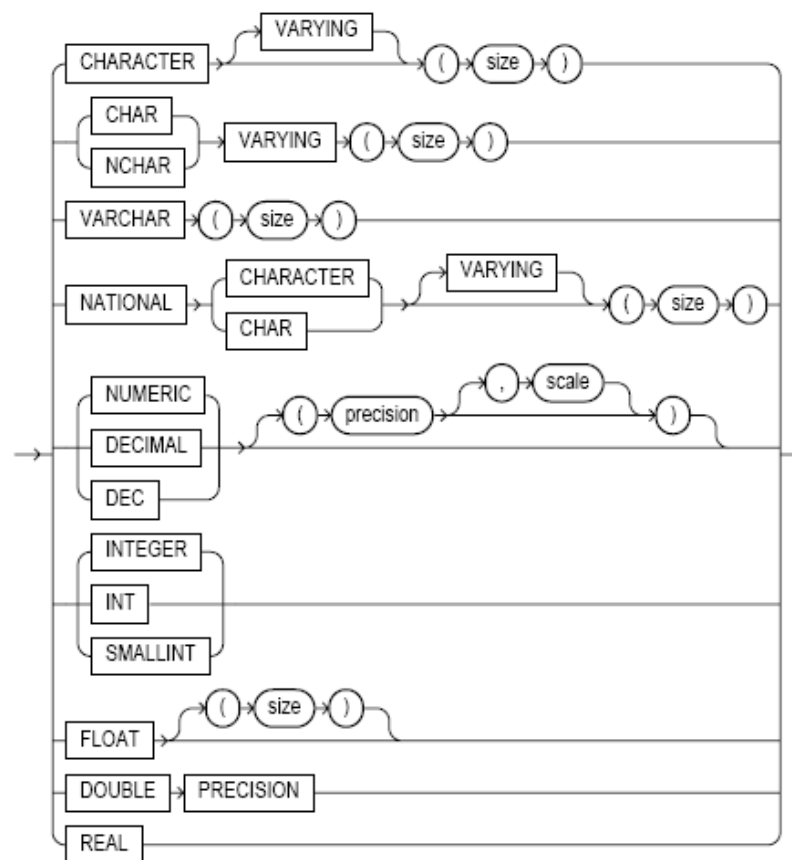


Types de données

ORACLE

Les autres types
supportés par
ORACLE

ANSI_supported_datatypes::=





Constantes



- Les **constantes numériques** sont spécifiées avec les chiffres **0** à **9**, un **point décimal** si nécessaire et le symbole **+** ou **-** au début; on peut aussi utiliser la lettre **E** pour les valeurs en notation scientifique
 - **7, -1, +24.5, -4.43, 6.63E23, 4.19E-31**
- Les **constantes chaîne de caractères** sont encadrées avec des **'apostrophes'**
 - **'Allo', 'Jacques Cartier', 'Aujourd'hui', '21-JAN-2007'**



EXERCICES #1

ORACLE

- Traduire la description d'une entité conceptuelle en table à l'aide du SQL (LDD) et de la commande CREATE



Destruction d'une table **ORACLE**

- L'énoncé **DROP TABLE** sert à détruire une table dans le schéma de l'utilisateur. Sous réserve de certains droits, on pourrait aussi détruire les tables qui appartiennent à un autre utilisateur. (DROP ANY TABLE)
- Lorsqu'on détruit une table, et si la table que l'on souhaite détruire est liée à une autre table du modèle par une relation, deux solutions sont possibles :
 - La clause **CASCADE CONSTRAINTS** pour détruire les objets dépendants de cette table (en mode cascade)
 - La contrainte qui permet de faire le lien avec la deuxième table est désactivée (isolée), puis la table est supprimée



Destruction d'une table **ORACLE**

- L'instruction **DROP TABLE** entraîne la suppression des données, de la structure, de la description dans le dictionnaire des données, des index, des déclencheurs associés et la récupération de l'espace de stockage *Pages 37-38 Livre C. Soutou*



Destruction d'une table ORACLE

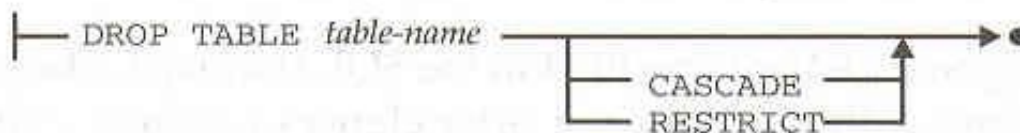


Figure 13-3. *DROP TABLE statement syntax diagram*

EXEMPLES :

DROP TABLE EX1_VENDEUR_VEN CASCADE CONSTRAINTS;

DROP TABLE JOSEPH.ANNIVERSAIRE;



Table - Contraintes

ORACLE

- Les contraintes exprimées sur les données assurent la **cohérence** et la **pertinence** des données contenues dans une base de données.
- Quatre types de **contraintes** :

```
CONSTRAINT nomContrainte
```

- **UNIQUE** (*colonne1* [, *colonne2*]...)
- **PRIMARY KEY** (*colonne1* [, *colonne2*]...)
- **FOREIGN KEY** (*colonne1* [, *colonne2*]...)
 REFERENCES [*schéma.*]*nomTablePere* (*colonne1* [, *colonne2*]...)
 [**ON DELETE** { **CASCADE** | **SET NULL** }]
- **CHECK** (*condition*)

*SQL pour Oracle, C. Soutou,
Édition 5, P40*



Table - Contraintes

ORACLE

- Les **contraintes de portée table** : contraintes qui portent sur une ou plusieurs colonnes pour en assurer l'unicité, la validation et l'intégrité référentielle
 - Clés primaires (Primary Key)
 - Clés étrangères (Foreign Key)
 - Clé unique

```
CREATE TABLE Compagnie  
  (comp CHAR(4), nrue NUMBER(3),  
   rue CHAR(20), ville CHAR(15) DEFAULT 'Paris',  
   nomComp CHAR(15) NOT NULL,  
   CONSTRAINT pk_Compagnie PRIMARY KEY(comp));
```

Deux contraintes en ligne et une
contrainte nommée de clé primaire.



Contraintes de table clés primaires et étrangères

ORACLE

- **Contrainte de clé primaire** peut être déclarée:
 - **Clé primaire simple** → au niveau table ou au niveau colonne
 - **Clé primaire composée** → au niveau table
- **Contrainte « clé étrangère »** peut être déclarée:
 - **Clé étrangère simple** → de niveau colonne ou table
 - **Clé étrangère composée** → de niveau table



Contraintes de table clés primaires et étrangères

ORACLE

- Il est fortement **recommandé** d'utiliser la syntaxe avec le mot clé **CONSTRAINT** afin de pouvoir nommer les contraintes (*Pages 24-25 de votre livre*).
- Donner un nom à une **contrainte** permet d'avoir un contrôle sur cette dernière, l'activer, la désactiver, la supprimer (*Voir page 87-89 de votre livre*).



Contraintes de table

ORACLE

- Lorsqu'au cours d'un ordre SQL d'**insertion**, de **modification** ou de **suppression**, une contrainte n'est pas vérifiée on dit qu'il y a "violation" de la contrainte et les effets de l'ordre SQL sont totalement annulé (**ROLLBACK**).



Contraintes de table clés primaires et étrangères

ORACLE

Exemple de
contrainte de *clé
primaire*

Exemples de
contraintes de *clé
étrangère*

```
CREATE TABLE ORDERS
  (ORDER_NUM INTEGER NOT NULL,
   ORDER_DATE DATE NOT NULL,
   CUST INTEGER NOT NULL,
   REP INTEGER,
   MFR CHAR(3) NOT NULL,
   PRODUCT CHAR(5) NOT NULL,
   QTY INTEGER NOT NULL,
   AMOUNT MONEY NOT NULL,
   PRIMARY KEY (ORDER_NUM),
   CONSTRAINT PLACEDBY
   FOREIGN KEY (CUST)
   REFERENCES CUSTOMERS
   ON DELETE CASCADE,
   CONSTRAINT TAKENBY
   FOREIGN KEY (REP)
   REFERENCES SALESREPS
   ON DELETE SET NULL,
   CONSTRAINT ISFOR
   FOREIGN KEY (MFR, PRODUCT)
   REFERENCES PRODUCTS
   ON DELETE RESTRICT)
```




Contraintes de table clés primaires et étrangères

ORACLE

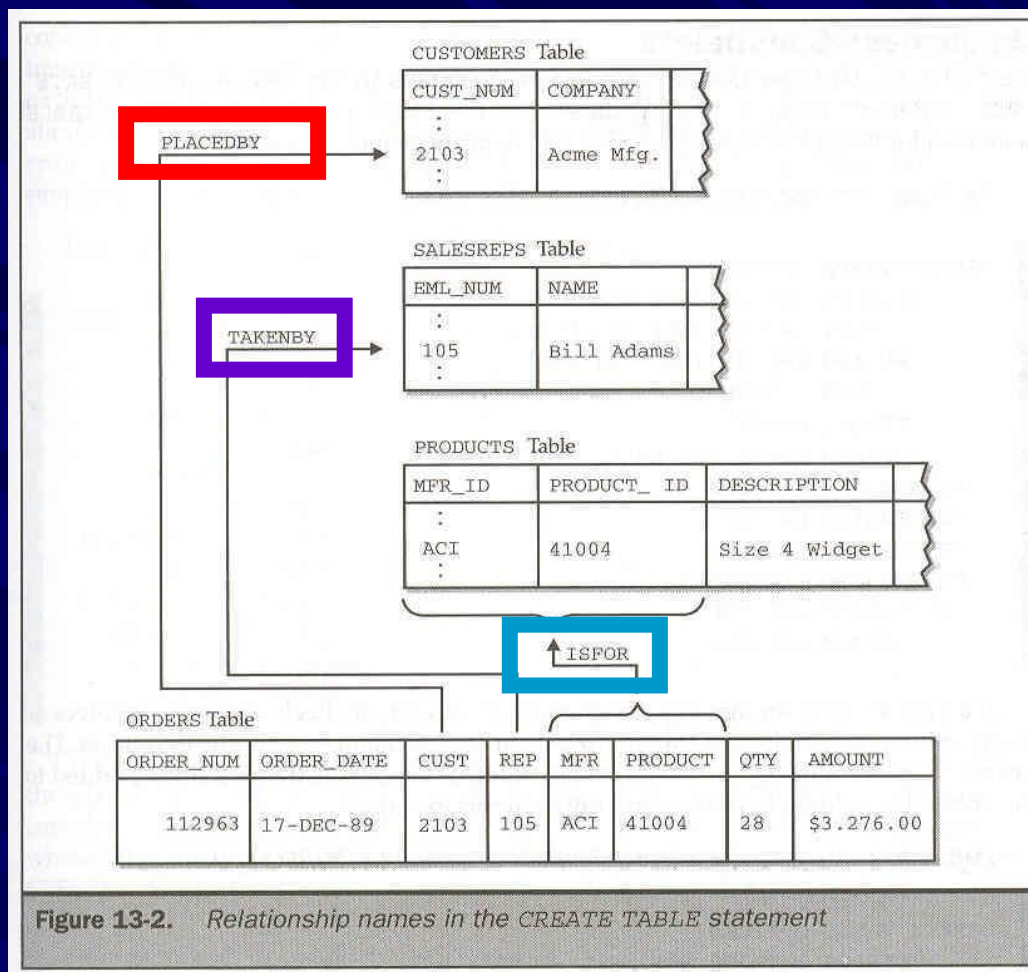


Figure 13-2. Relationship names in the CREATE TABLE statement



Création d'une table

ORACLE

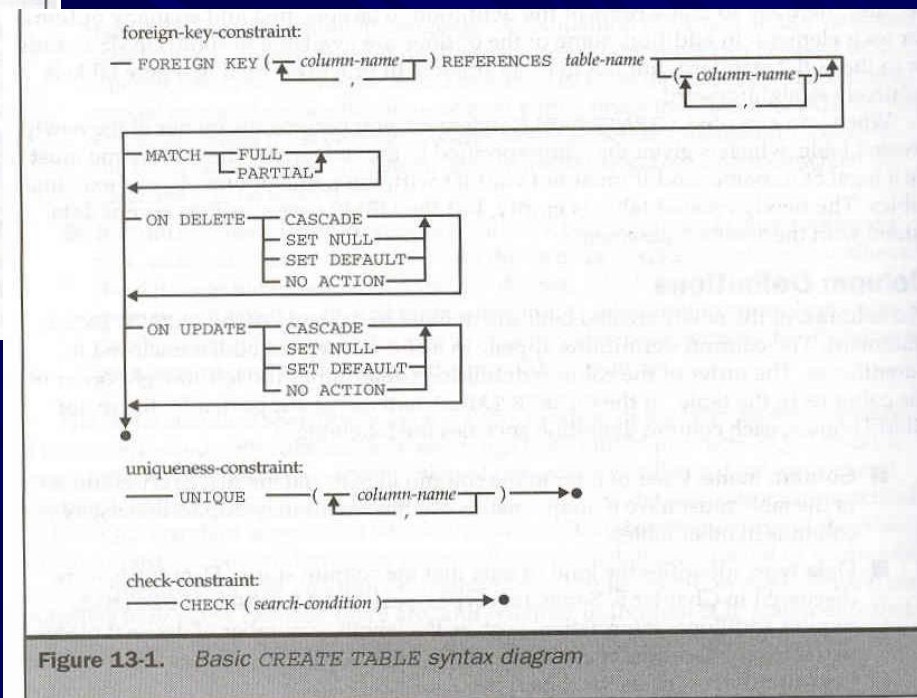
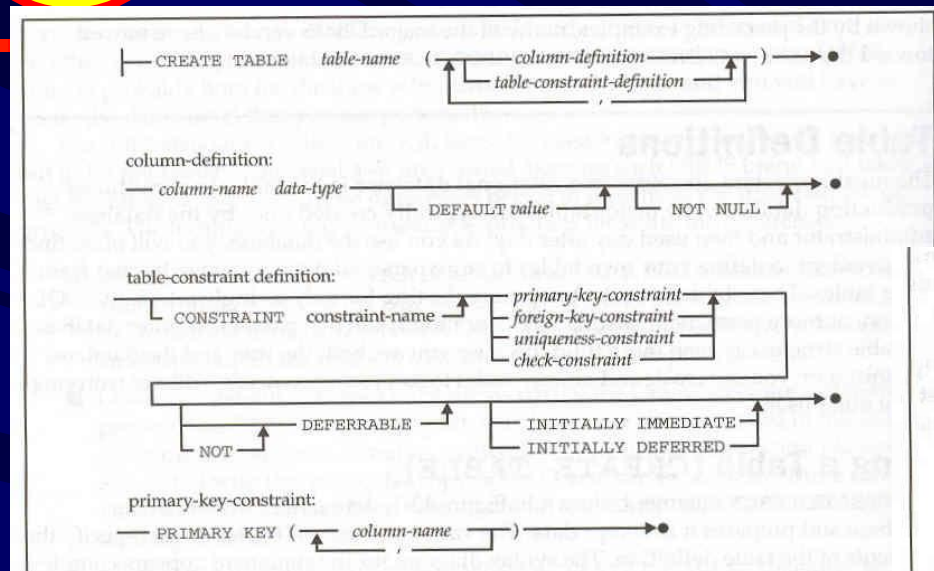


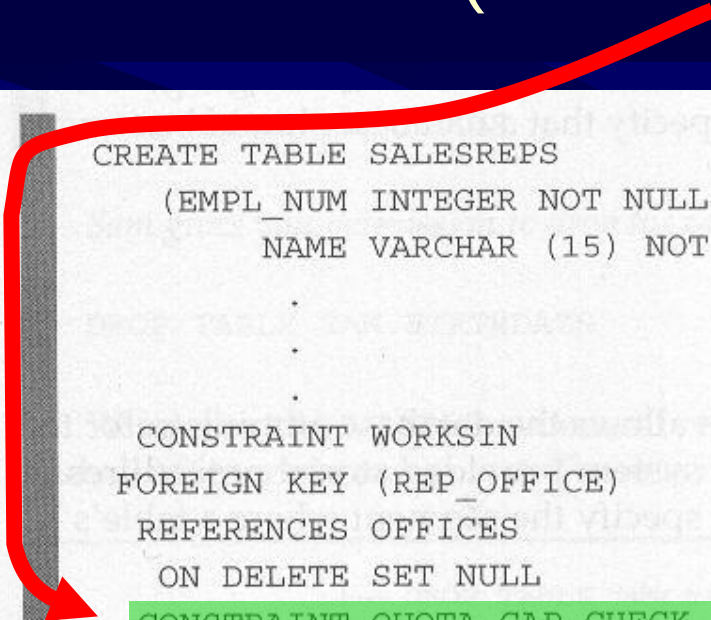
Figure 13-1. Basic CREATE TABLE syntax diagram



Contraintes de colonne de type validation

ORACLE

Une contrainte du genre **CHECK** applique une règle de validation (de niveau entité).



```
CREATE TABLE SALESREPS
  (EMPL_NUM INTEGER NOT NULL,
   NAME VARCHAR (15) NOT NULL,
   .
   .
   .
  CONSTRAINT WORKSIN
  FOREIGN KEY (REP_OFFICE)
  REFERENCES OFFICES
  ON DELETE SET NULL
  CONSTRAINT QUOTA_CAP CHECK ((HIRE_DATE < "01-JAN-88") OR
                               (QUOTA <= 300000)))
```



Contraintes de colonne de type unique

ORACLE

On peut appliquer une contrainte d'unicité (**UNIQUE**) qui force un champ à assumer des valeurs distinctes (uniques) même si ce champ n'est pas la clé primaire! (clé candidate)

```
CREATE TABLE OFFICES
(OFFICE INTEGER NOT NULL,
 CITY VARCHAR(15) NOT NULL,
 REGION VARCHAR(10) NOT NULL,
 MGR INTEGER,
 TARGET MONEY,
 SALES MONEY NOT NULL,
 PRIMARY KEY (OFFICE),
 CONSTRAINT HASMGR
 FOREIGN KEY (MGR)
 REFERENCES SALESREPS
 ON DELETE SET NULL,
 UNIQUE (CITY))
```




Contrainte de colonne CHECK

ORACLE

```
CREATE TABLE Tab_contraintes (  
  colonne1 NUMBER(5)    CONSTRAINT pk_colonne1 PRIMARY KEY,  
  colonne2 VARCHAR2(30) CONSTRAINT nn_colonne2 NOT NULL,  
  colonne3 NUMBER(2)    CONSTRAINT ck_colonne3 CHECK (colonne3 > 10),  
  colonne4 CHAR(1)      CONSTRAINT ck_colonne4 CHECK (colonne4 IN ('A', 'B', 'C')),  
  colonne5 VARCHAR2(50) CONSTRAINT ck_colonne5 CHECK (colonne5 LIKE 'M%')  
;  
  
ALTER TABLE Tab_contraintes ADD CONSTRAINT nn_colonne4 CHECK (colonne4 IS NOT NULL);
```



Contrainte UNIQUE

ORACLE

- La contrainte UNIQUE identifie de façon unique chaque enregistrement dans une table de base de données.
- Si une ou plusieurs colonnes de tables est associé à une contrainte UNIQUE, cela exige que chaque valeur d'une colonne ou de l'ensemble des colonnes (si plus qu'une colonne) doit être unique;

```
CONSTRAINT clients_phone_email_uk UNIQUE(email,phone)
```



Contrainte UNIQUE

ORACLE

- À la différence de la clé primaire, la contrainte UNIQUE autorise la saisie de valeur NULL sauf si la colonne a également une contrainte NOT NULL défini.
- Une colonne (ou un groupe de colonnes) avec la valeur NULL n'empêche pas une contrainte UNIQUE car les valeurs NULL ne sont pas considérées comme égales à quoi que ce soit.



Contrainte UNIQUE

ORACLE

- À la différence de la clé primaire, la contrainte UNIQUE autorise la saisie de valeur NULL sauf si la colonne a également une contrainte NOT NULL défini.
- Une colonne (ou un groupe de colonnes) avec la valeur NULL n'empêche pas une contrainte UNIQUE car les valeurs NULL ne sont pas considérées comme égales à quoi que ce soit.



Contrainte UNIQUE

ORACLE

CLIENT_NUMBER	FIRST_NAME	LAST_NAME	PHONE	EMAIL
5922	Hiram	Peters	3715832249	hpeters@yahoo.com
5857	Serena	Jones	7035335900	serena.jones@jones.com
6133	Lauren	Vigil	4072220090	lbv@lbv.net
7234	Lonny	Vigil	4072220091	lbv@lbv.net



La combinaison de valeurs doit être unique



Règles de nommage

ORACLE

- **Contrainte** → **Préfixe_Schéma_Nom1_Nom2**
 - **Préfixe** → (Page 41 Livre C. Soutou)
 - **PK_** (clé primaire)
 - **FK_** (clé étrangère)
 - **NN_** (NOT NULL)
 - **CK_** (règle de validité ⇔ check)
 - **UN_** (valeur unique)
 - **DK_** (valeur par défaut)
 - **Schéma** → Schéma relationnel (3 car.)
 - **Nom1** → Suffixe de la *table actuelle* (3 car.)
 - **Nom2** →
 - Contraction du nom de l'attribut ...ou...
 - Suffixe de la *table mère* si c'est une *clé étrangère* (3 car.)



Atelier rencontre 04

ORACLE

- **Réaliser l'atelier de la rencontre dans les délais et déposez-le sur LEA**



Modification d'une table

ORACLE

Chapitre 03 C. Soutou (P. 77-100)

- L'énoncé **ALTER TABLE** sert à modifier la structure d'une table :
 - Ajouter (**ADD**) une ou plusieurs colonnes, une contrainte
 - Modifier (**MODIFY**) la description d'une ou plusieurs colonnes (type, contrainte)
 - Supprimer (**DROP**) une colonne, une contrainte
 - Renommer (**RENAME**) une colonne (depuis la version 9i)
 - Renommer une table



Modification d'une table

Chapitre 03 C. Soutou (P. 77-100)

ORACLE

- Avec Oracle, pour pouvoir modifier la structure d'une table, vous devez disposer des privilèges qu'il faut :
 - Vous devez être celui qui a créé la table
 - Vous avez le privilège **ALTER TABLE**
- Lorsqu'un nouveau champ est ajouté à une table, les valeurs assumées par ce champ sont normalement des valeurs **NULL** sauf si la clause **DEFAULT** est spécifiée.
- Lorsqu'une **contrainte** est ajoutée, par défaut cette dernière est **activée**.



Modification d'une table

ORACLE

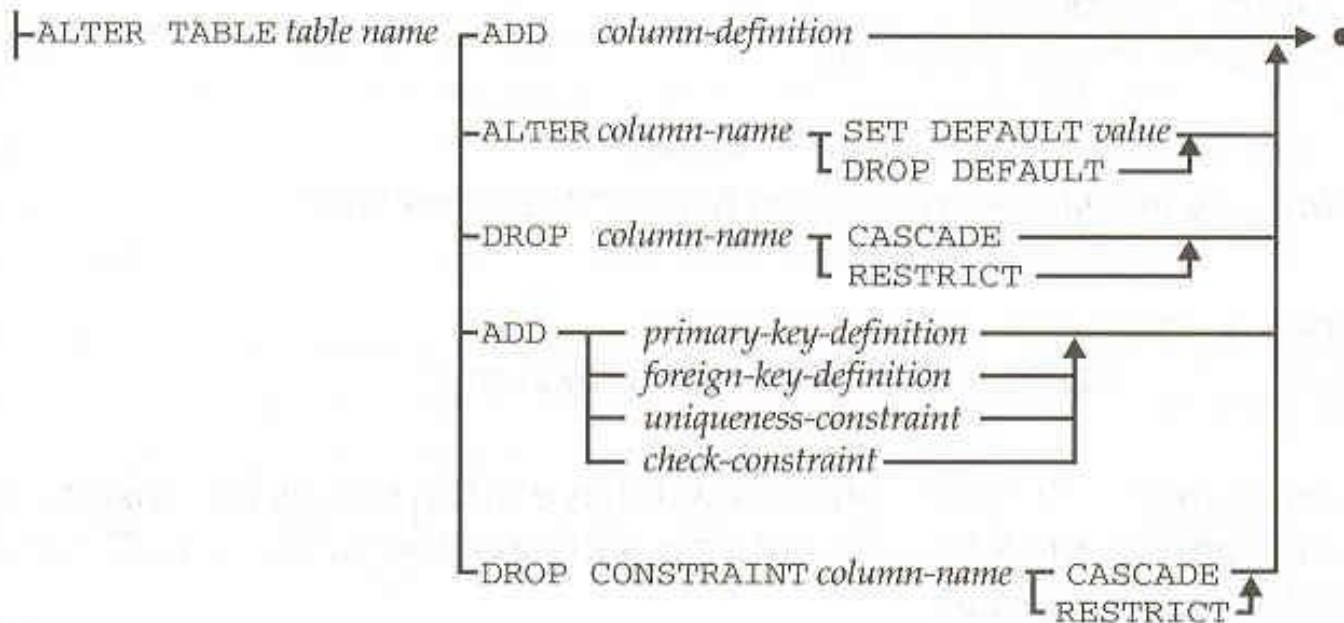


Figure 13-4. *ALTER TABLE* statement syntax diagram



Modifier la structure d'une table

ORACLE

- Ajouter une ou plusieurs colonnes

```
ALTER TABLE NomTable  
  ADD (nomColonne NUMBER(7));  
  
ALTER TABLE NomTable  
  ADD (nomColonne  NUMBER(7) DEFAULT 'GG',  
       nomColonne2 VARCHAR2(20) NOT NULL);
```

- Modifier le type de donnée d'une colonne

```
ALTER TABLE NomTable  
  MODIFY id NUMBER(7) DEFAULT 10;  
  
ALTER TABLE NomTable  
  MODIFY id NUMBER(7) NOT NULL;
```



Modifier la structure d'une table

ORACLE

- **Supprimer une colonne**

```
ALTER TABLE NomTable  
  DROP COLUMN nomColonne ;  
  
ALTER TABLE NomTable  
  SET UNUSED COLUMN nomColonne ;
```

- **Contraintes d'utilisation avec Oracle** : il n'est pas possible d'appliquer une suppression sur :
 - Une clé primaire (ou candidate) si elle est référencée par une clé étrangère
 - Une colonne à partir duquel un index a été construit
 - Des pseudo-colonnes (ROWID et LEVEL) ou des colonnes de tables de type objets.



Modifier la structure d'une table

ORACLE

- **Ajouter une contrainte** : Il est d'usage de créer les tables sans leurs contraintes dans un script et d'écrire un autre script pour implanter les contraintes de ces tables.

```
ALTER TABLE Vente ADD CONSTRAINT nn_prix CHECK (prix IS NOT NULL),  
                CONSTRAINT fk_clt_Client FOREIGN KEY(clt) REFERENCES Client(code);
```



Modifier la structure d'une table

ORACLE

- La contrainte est par défaut activée lors de sa création
- Supprimer ou désactiver une contrainte

```
ALTER TABLE NomTable  
    DROP CONSTRAINT nomContrainte ;  
ALTER TABLE NomTable  
    DROP CONSTRAINT nomContrainte CASCADE;  
ALTER TABLE NomTable  
    DISABLE CONSTRAINT nomContrainte ;  
ALTER TABLE NomTable  
    ENABLE CONSTRAINT nomContrainte ;
```



Modifier la structure d'une table

ORACLE

- Exercice 03 : modifier la structure d'une table pour :
 - Ajouter une colonne
 - Modifier traduire la description logique d'une table à l'aide du LDD et de la commande CREATE